

UNIVERSITÀ DEGLI STUDI DI VERONA

LAUREA IN INFORMATICA

TESI DI LAUREA

SIMULAZIONE DEI FLUIDI IN TEMPO REALE TRAMITE GPU E VULKAN

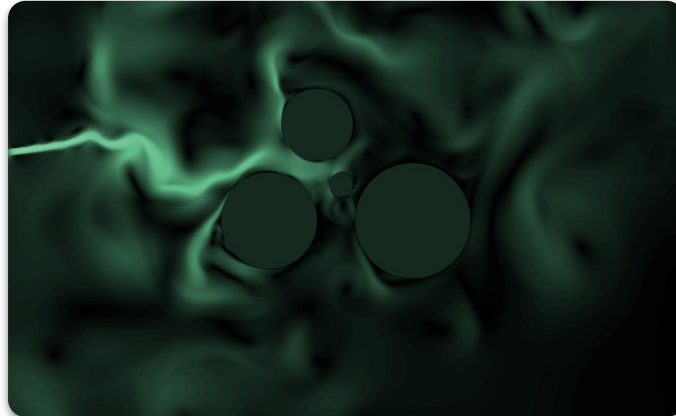
Relatrice: Prof.ssa Claudia Daffara

Laureando: Andrea Missaggia VR389742

ANNO ACCADEMICO 2023-2024

Obiettivi della tesi

- Approfondire **Stable Fluids**, una tecnica di simulazione dei fluidi proposta da **Joe Stam (1999)** e basata sulle equazioni di **Navier-Stokes**.
- Proporre un'implementazione moderna basata su **API Vulkan** e **compute-shader**.
- Sfruttare le capacità di calcolo parallelo della **GPU** per migliorare efficienza e prestazioni.



Caratteristiche dell'algoritmo

- **Fluidi incompressibili con densità costante:** ipotesi semplificatrice.
- **Metodo della proiezione:** per semplificare le equazioni di Navier-Stokes.
- **Metodo delle differenze finite:** per calcolare numericamente le derivate.
- **Metodo di Jacobi:** per risolvere iterativamente sistemi di equazioni lineari.
- **Metodo implicito:** per garantire stabilità nel calcolo del futuro stato del fluido.

Caratteristiche dell'implementazione

- **API Vulkan:** framework per il rendering e il calcolo ad alte prestazioni.
- **Immagine bidimensionale:** per memorizzare lo stato del campo di velocità e di pressione.
- **Meccanismi di sincronizzazione:** per coordinare le operazioni tra CPU $\Leftrightarrow$ GPU e GPU $\Leftrightarrow$ GPU.
- **Compute shader:** per implementare ed eseguire i vari step della simulazione sulla GPU.

Risorse del progetto

- **Sorgenti del progetto:**

<https://github.com/AndreaMissaggia/Fluid-Simulation>

- **Video dimostrativo:**

<https://github.com/AndreaMissaggia/Fluid-Simulation/tree/main/video>

Equazioni di Navier-Stokes per fluidi incomprimibili

- Sistema di equazioni che descrive il moto del fluido:

$$1) \frac{\partial \mathbf{u}}{\partial t} = - (\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \qquad 2) \nabla \cdot \mathbf{u} = 0$$

Dove:

- **$\mathbf{u}$ velocità del fluido** (campo vettoriale)
- **p pressione interna al fluido** (campo scalare)
- **$\mathbf{f}$ forze esterne applicate al fluido** (vettore)
- **ρ densità del fluido**
- **ν costante di viscosità del fluido**
- **t tempo**

- La simulazione considera un campo bidimensionale, di conseguenza $\mathbf{u}$ ha due componenti:

$$\mathbf{u}(x, y, t) = (u(x, y, t), v(x, y, t))$$

Equazioni di Navier-Stokes per fluidi incomprimibili

$$1) \frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \quad \rightarrow \quad \begin{aligned} \frac{\partial u}{\partial t} &= -(\mathbf{u} \cdot \nabla) u - \frac{1}{\rho} \nabla p + \nu \nabla^2 u + f \\ \frac{\partial v}{\partial t} &= -(\mathbf{u} \cdot \nabla) v - \frac{1}{\rho} \nabla p + \nu \nabla^2 v + f \end{aligned}$$

- La seconda equazione ($\nabla \cdot \mathbf{u} = 0$) è chiamata **equazione di continuità** e garantisce che la velocità del fluido abbia sempre divergenza nulla. Questo, come vedremo, implica che il fluido sia effettivamente incomprimibile.

In totale, nel sistema, ci sono tre equazioni e tre incognite: u , v , e p .

Termini delle Equazioni di Navier Stokes

I termini a destra della prima equazione di Navier-Stokes rappresentano accelerazioni; esprimono un cambiamento della velocità del fluido nel tempo.

$-(\mathbf{u} \cdot \nabla) \mathbf{u}$	$-\rho^{-1} \nabla p$	$+\nu \nabla^2 \mathbf{u}$	$+\mathbf{f}$
<i>avvezione</i>	<i>pressione</i>	<i>diffusione</i>	<i>forze esterne</i>

- **Avvezione:** le particelle del fluido si muovono nel campo e trasportano con sé la loro velocità. Il segno negativo indica una perdita di velocità: è la velocità che lascia quel punto del campo perché trasportata altrove.
- **Pressione:** differenze di pressione inducono accelerazioni, il fluido tende a spostarsi da regioni ad alta pressione a quelle a bassa pressione; il fluido tende ad accelerare in direzione opposta al gradiente della pressione. Per semplicità consideriamo un fluido con densità costante $\rho = 1$.
- **Diffusione:** la viscosità del fluido genera resistenza al moto delle particelle; le particelle si scambiano energia cinetica uniformando gradualmente la velocità.
- **Forze esterne:** ad esempio la forza di gravità applicata al fluido.

Rappresentazione del campo di velocità e di pressione del fluido

Campo del fluido rappresentato attraverso un **array bidimensionale** (immagine):

- Ogni cella (pixel) contiene un valore del campo di velocità e del campo di pressione del fluido:

Valori del campo codificati nelle componenti del colore dei pixel (RGB).

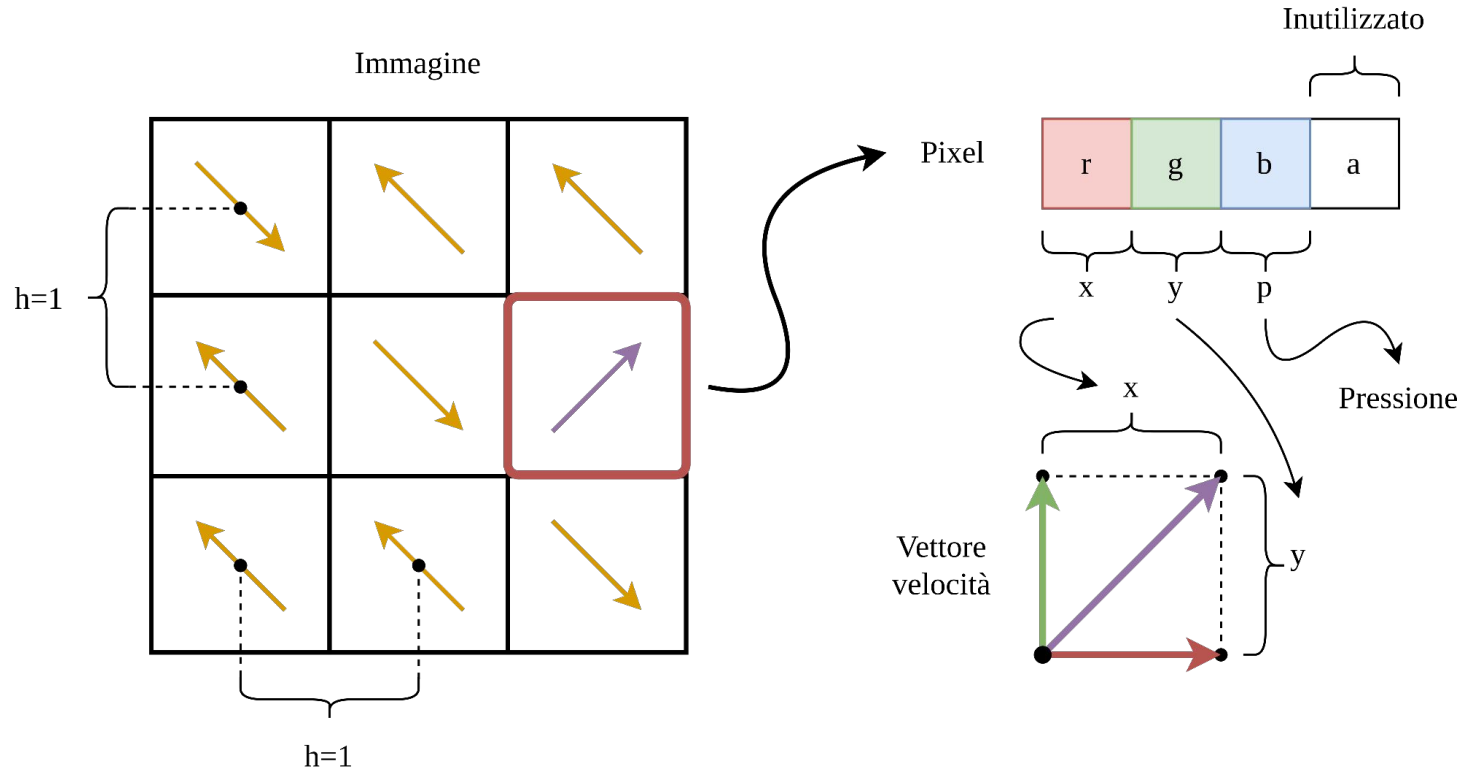
- Il dominio continuo viene discretizzato per applicare metodi numerici:

Il campo viene valutato su una griglia bidimensionale.

Passo di discretizzazione (h):

- Distanza tra i centri di due celle adiacenti (distanza tra due punti adiacenti del dominio discreto).
- Impiegato nel calcolo approssimato delle derivate.
- **Il passo della griglia è lo stesso in verticale e in orizzontale** (i pixel sono quadrati).
- Deve essere un **numero intero** per accedere correttamente agli elementi di un array.
- Ponendo $h=1$, si garantisce lo step più piccolo possibile.

Rappresentazione del campo di velocità e di pressione del fluido (visione Euleriana)



Metodo delle differenze finite

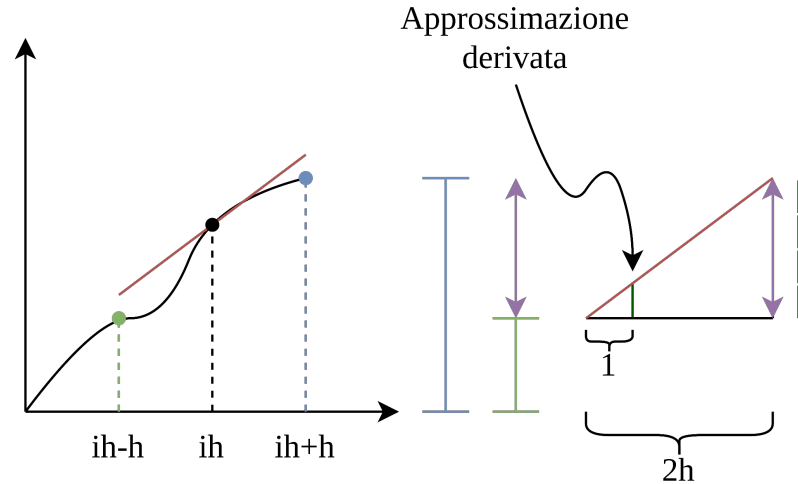
- Tecnica numerica per approssimare il calcolo delle **derivate**.
- Sostituisce il calcolo del limite con **differenze finite**.
- Stam utilizza questa tecnica per discretizzare gli **operatori differenziali**.

Nota: in un campo discreto l'operazione di limite non è definita.

Metodo della differenza finita centrata

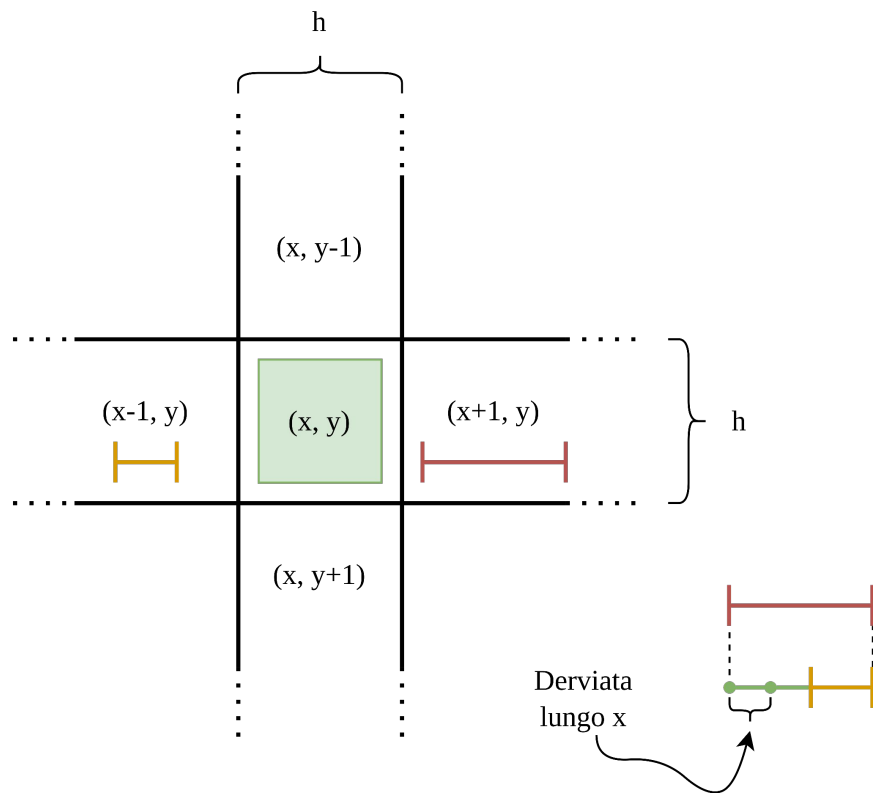
Tecnica per approssimare numericamente la derivata di una funzione f di cui sono noti i valori su un insieme discreto di punti:

$$f'(ih) \approx \frac{f(ih+h) - f(ih-h)}{2h}$$



Dove h è la distanza tra due punti discreti su cui la funzione è campionata e ih indica la posizione del punto i -esimo, dove i è un indice intero ($i = 0, 1, 2, \dots, N$).

Approssimazione delle derivate parziali del campo discreto



Grazie al **metodo della differenza finita centrata** e al passo di discretizzazione **$h=1$** , il calcolo delle derivate parziali del campo discreto per una cella (x, y) , si riduce sostanzialmente alla differenza dei valori delle celle adiacenti. Considerando un campo scalare p :

Derivata lungo x:

$$\frac{\partial p(x, y)}{\partial x} \approx \frac{p(x+h, y) - p(x-h, y)}{2h} = \frac{p(x+1, y) - p(x-1, y)}{2}$$

Derivata lungo y:

$$\frac{\partial p(x, y)}{\partial y} \approx \frac{p(x, y+h) - p(x, y-h)}{2h} = \frac{p(x, y+1) - p(x, y-1)}{2}$$

Approssimazione operatori differenziali

Una volta definita l'approssimazione delle derivate parziali, è facile ottenere un'approssimazione per altri operatori differenziali.

Gradiente di un campo scalare p :

$$\nabla p(x, y) = \left(\frac{\partial p(x, y)}{\partial x}, \frac{\partial p(x, y)}{\partial y} \right) \approx \left(\frac{p(x+1, y) - p(x-1, y)}{2}, \frac{p(x, y+1) - p(x, y-1)}{2} \right)$$

Divergenza di un campo vettoriale $\mathbf{u}(u(x, y), v(x, y))$:

$$\nabla \cdot \mathbf{u}(x, y) = \frac{\partial u(x, y)}{\partial x} + \frac{\partial v(x, y)}{\partial y} \approx \frac{u(x+1, y) - u(x-1, y)}{2} + \frac{v(x, y+1) - v(x, y-1)}{2}$$

Metodo dello schema a cinque punti

Il metodo dello schema a cinque punti, noto in inglese come *Five-point stencil*, è una **tecnica delle differenze finite** utilizzata per approssimare operatori differenziali su griglie bidimensionali.

L'operatore di Laplace viene approssimato utilizzando questo metodo:

$$\nabla^2 p(x, y) = \nabla \cdot (\nabla p(x, y)) \approx \frac{p(x+h, y) + p(x-h, y) + p(x, y+h) + p(x, y-h) - 4p(x, y)}{h^2}$$

Dove p è una funzione scalare e h è la distanza tra i punti discreti, ovvero la lunghezza del lato di una cella.

Nel contesto della simulazione, dove consideriamo $h=1$, **l'approssimazione diventa:**

$$\nabla^2 p(x, y) \approx \frac{p(x+1, y) + p(x-1, y) + p(x, y+1) + p(x, y-1) - 4p(x, y)}{1}$$

Introduzione a Stable Fluids

L'algoritmo proposto da Stam si basa sul **metodo della proiezione**, noto anche come metodo di proiezione di Chorin.

Metodo di Proiezione (Chorin, 1968):

- Originariamente introdotto per risolvere numericamente le equazioni di Navier-Stokes.
- Basato sul teorema di *decomposizione di Helmholtz-Hodge*.
- Impiegato da Stam per semplificare il sistema di equazioni.

Teorema di Helmholtz-Hodge:

- Un campo vettoriale $\mathbf{w}$ può essere scomposto nella somma di due componenti: $\mathbf{w} = \mathbf{u} + \nabla q$
- $\mathbf{u}$ è la componente solenoidale, un campo vettoriale con divergenza nulla.
- ∇q è la componente irrotazionale, il gradiente di un campo scalare.

Introduzione a Stable Fluids

L'algoritmo di Stam è essenzialmente un *approccio di separazione degli operatori* (operator splitting), il calcolo dell'avvezione, delle forze viscosi e delle forze esterne viene separato dal contributo delle forze di pressione:

1. Nella prima fase viene calcolata una **velocità intermedia** che non soddisfa il vincolo di incomprimibilità.
 2. Nella seconda fase viene utilizzata la pressione per **proiettare** il campo di velocità nella sua componente con divergenza nulla.
- Stam definisce a questo scopo un operatore **P** di proiezione:

$$\mathbf{P}(\mathbf{w}) = \mathbf{w} - \nabla q = \mathbf{u}$$

Introduzione a Stable Fluids - Operatore di proiezione

L'operatore $\mathbf{P}$ proietta un qualsiasi campo vettoriale $\mathbf{w}$ nella sua componente con divergenza nulla:

$$\mathbf{P}(\mathbf{w}) = \mathbf{w} - \nabla q = \mathbf{u}$$

In altri termini, $\mathbf{P}$ sottrae la componente “responsabile” della divergenza dal campo.

- Come vedremo in seguito, Stam utilizza questo risultato e il gradiente della pressione per correggere il campo di velocità del fluido in modo da ottenere, ad ogni step della simulazione, un campo di velocità con divergenza nulla e che, quindi, soddisfa l'equazione di continuità.

Introduzione a Stable Fluids

Stam utilizza l'operatore di proiezione per semplificare le equazioni di Navier-Stokes, applicando l'operatore di proiezione a entrambi i membri della prima equazione del sistema si ottiene infatti la seguente equazione:

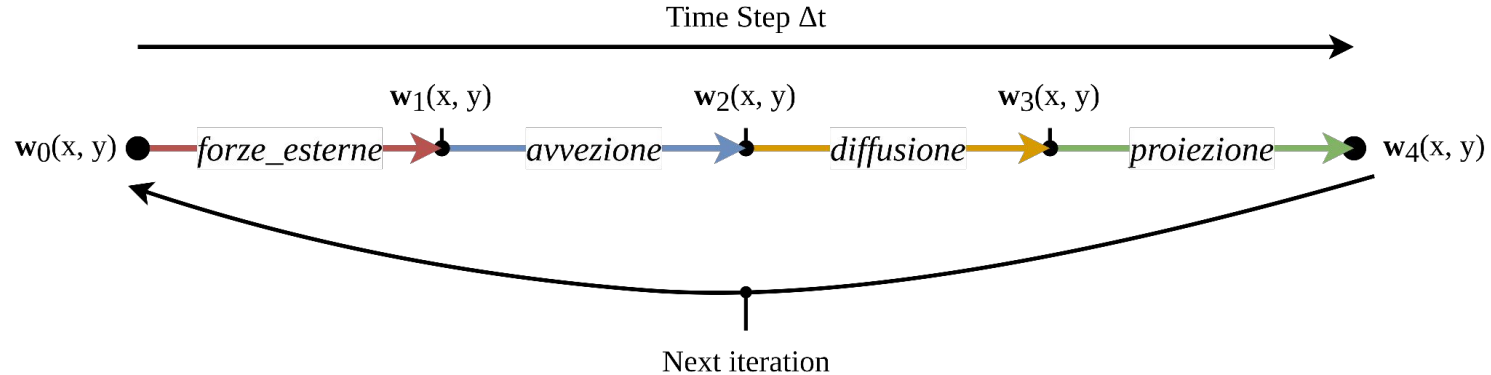
$$\frac{\partial \mathbf{u}}{\partial t} = \mathbf{P}(- (\mathbf{u} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f})$$

- $\mathbf{P}(\mathbf{u}) = \mathbf{u}$ (equazione di continuità, proietto $\mathbf{u}$ nella sua componente con divergenza nulla).
- $\mathbf{u}$ ha divergenza nulla di conseguenza anche la sua derivata ha divergenza nulla.
- $\mathbf{P}(\nabla p) = 0$ in quanto è la componente con divergenza nulla del gradiente di un campo scalare.

Questa equazione rappresenta simbolicamente l'intero algoritmo utilizzato per simulare il comportamento dei fluidi.

Simulazione - Iterazione

Stam divide e risolve singolarmente, in sequenza, i termini contenuti tra parentesi nella parte destra dell'equazione, infine corregge la divergenza del campo utilizzando l'operazione di proiezione. Ogni termine rappresenta un'operazione diversa che modifica il campo di velocità del fluido:



- $w_0(x, y) \approx \mathbf{u}(x, y, t)$ rappresenta il campo di velocità del fluido al **time step i**.
- $w_4(x, y) \approx \mathbf{u}(x, y, t + \Delta t)$ rappresenta il campo di velocità del fluido al **time step i+1**.

Simulazione

- Stam prima calcola l'effetto sulla velocità causato dalle forze esterne, dell'avvezione e della diffusione senza considerare esplicitamente la pressione; nell'ultimo passaggio, utilizza il gradiente della pressione e l'operazione di proiezione per correggere la velocità del fluido in modo che mantenga divergenza nulla.
- La simulazione è ottenuta iterando questi step, ciascun passaggio verrà implementato come un compute-shader separato; i compute-shader verranno eseguiti in sequenza e opereranno su un set di immagini condiviso.
- Dividere le equazioni di Navier-Stokes in problemi più semplici permette a Stam di applicare una tecnica numerica specifica per ogni problema (divide et impera).
- Stam sfrutta la tecnica delle **differenze finite** per approssimare gli operatori differenziali coinvolti in queste operazioni.

Calcolo forze esterne

Il successivo stato del campo del fluido viene calcolato attraverso una composizione di funzioni:

$$\mathbf{w}_4(x, y) = \textit{proiezione}(\textit{diffusione}(\textit{avvezione}(\textit{forze_esterne}(\mathbf{w}_0(x, y)))))$$

Il primo termine calcolato è anche il più semplice da risolvere ed è il contributo delle forze esterne:

$$\mathbf{w}_1(x, y) = \mathbf{w}_0(x, y) + \frac{\mathbf{f}}{m} \cdot \Delta t$$

- Dove m è la massa del fluido contenuto in una cella. ($\rho = 1$ e $h = 1$ implica $m = 1$)

Pseudocodice:

$$\text{vel_next}[x, y] = \text{vel_current}[x, y] + \text{vector}(f_x, f_y) * 1/m * \text{delta_time}$$

Avvezione della velocità - Metodo esplicito

- Le particelle di un fluido si spostano e trasportano con sé la propria velocità attraverso il campo.
- Possiamo calcolare la nuova posizione delle particelle utilizzando la posizione attuale, la velocità attuale e il tempo trascorso.
- Possiamo quindi trasportare (avvezione) la velocità attraverso il campo fino al punto calcolato.

$$nuova_posizione_particelle = (x, y) + \mathbf{w}_1(x, y) \cdot \Delta t$$

Semplicemente, si usa la velocità attuale di una cella per determinare dove questa si sposterà. Questo metodo proietta il campo di velocità nello stato futuro, utilizzando la velocità attuale.

Calcolo avvezione della velocità - Metodo esplicito

Per ogni iterazione della simulazione:

delta_time: tempo trascorso tra due iterazioni della simulazione.

Per ogni cella/pixel di coordinate (x, y):

1. Calcolo vettore dello spostamento della velocità:

$$\text{vettore_spostamento} = \text{vel_current}[x, y] * \text{delta_time}$$

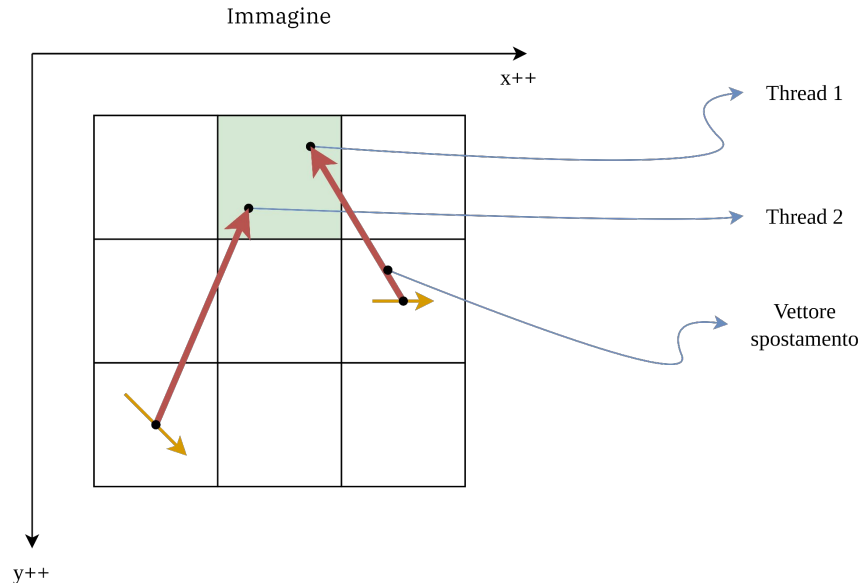
2. Aggiornamento del successivo valore del campo di velocità:

$$\text{vel_next}[\text{vector}(x, y) + \text{vettore_spostamento}] = \text{vel_current}[x, y]$$



$$\text{vel_next}[\text{vector}(x, y) + \text{vel_current}[x, y] * \text{delta_time}] = \text{vel_current}[x, y]$$

Problemi del metodo esplicito

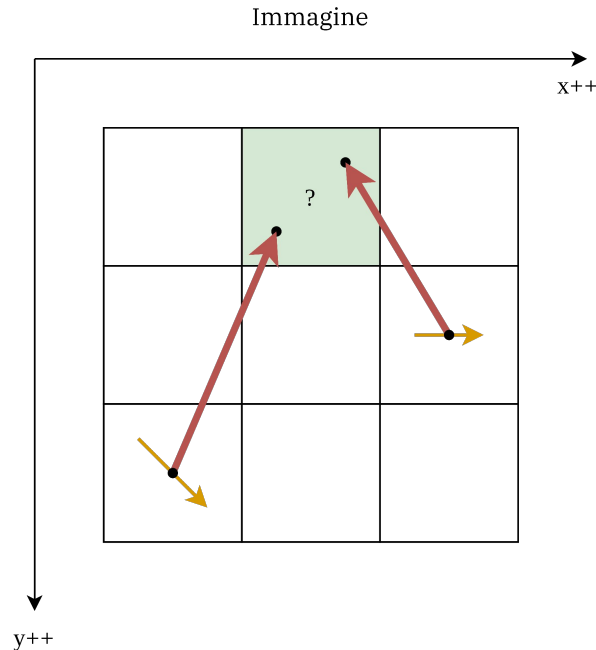


■ Primo problema, race condition in un contesto multi-thread:

Threads diversi potrebbero voler scrivere nella stessa cella contemporaneamente. Si potrebbe quindi generare una scrittura incoerente, dove il risultato finale dipende dall'ordine in cui i thread scrivono nella cella.

Per risolvere la race condition ogni thread dovrebbe poter scrivere esclusivamente nella cella che gli è stata assegnata, mentre sono concesse operazioni di letture su altre celle.

Problemi del metodo esplicito



■ Secondo problema, sovrapposizione dei vettori di spostamento:

Se i vettori di spostamento provenienti da celle diverse convergono nella stessa cella di destinazione, per calcolare la velocità finale di quella cella sarebbe necessario eseguire un'interpolazione tra tutte le velocità trasportate in quella cella, il cui numero è indefinito.

Problemi del metodo esplicito

■ Terzo problema, instabilità:

Per trovare la nuova posizione delle particelle sommiamo alla posizione attuale la derivata della posizione, ovvero la velocità, moltiplicata per il passo temporale. Questo metodo è noto come *metodo di Eulero*.

È facile intuire che se il delta temporale è troppo grande o il campo di velocità varia troppo rapidamente, il metodo di Eulero può portare a errori significativi (es. overshooting). Questo perché la derivata è un'approssimazione lineare della funzione, e in quanto tale è precisa solo in un intorno del punto in cui è calcolata. La velocità $\mathbf{u}$ è la derivata della posizione rispetto al tempo, la successiva posizione $\mathbf{x}$ delle particelle viene stimata applicando un passo del metodo di Eulero:

$$\frac{d\mathbf{x}}{dt} = \mathbf{u}(\mathbf{x}, t) \quad \mathbf{x}(t) = (x, y) \quad \mathbf{x}(t + \Delta t) \approx (x, y) + \mathbf{u}(x, y) \cdot \Delta t$$

Metodo di eulero

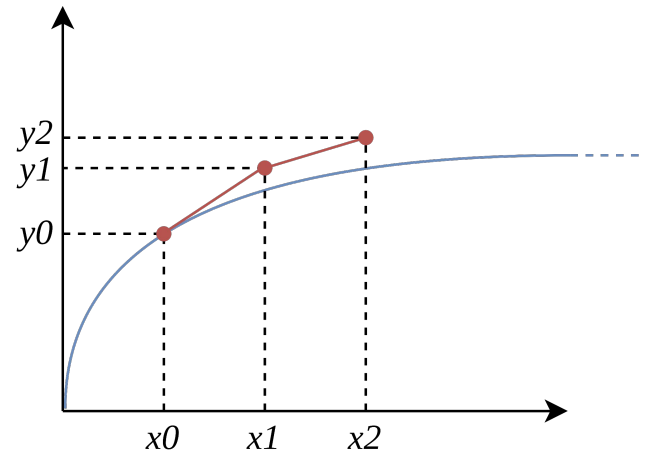
- Procedura numerica che ricava un'approssimazione della soluzione di un'equazione differenziale ordinaria partendo da un punto iniziale noto e dalla derivata della funzione incognita.

Il metodo di Eulero consente di stimare il successivo valore della funzione incognita su un dominio discreto, utilizzando la derivata e il valore calcolato al passo precedente:

$$y' = \frac{dy}{dx} = f(x, y)$$

$$y(x_0) = y_0 \text{ (punto iniziale noto)}$$

$$y_{i+1} = y_i + f(x_0 + hi, y_i) \cdot h \quad h \text{ passo del dominio discreto}$$



Avvezione della velocità - Metodo implicito

Stam utilizza un metodo semi-Lagrangiano per calcolare l'effetto dell'avvezione, questo metodo prevede di calcolare il punto di partenza delle particelle del fluido seguendo a ritroso la loro velocità attuale e di trasportare la quantità trovata nel punto di partenza nella posizione attuale. Questo approccio evita l'accumulo di errori e consente di utilizzare passi temporali più grandi risultando più stabile del metodo esplicito.

$$\mathbf{w}_2(x, y) = \mathbf{w}_1\left((x, y) - \mathbf{w}_1(x, y) \cdot \Delta t\right)$$

Calcolo avvezione della velocità - Metodo implicito

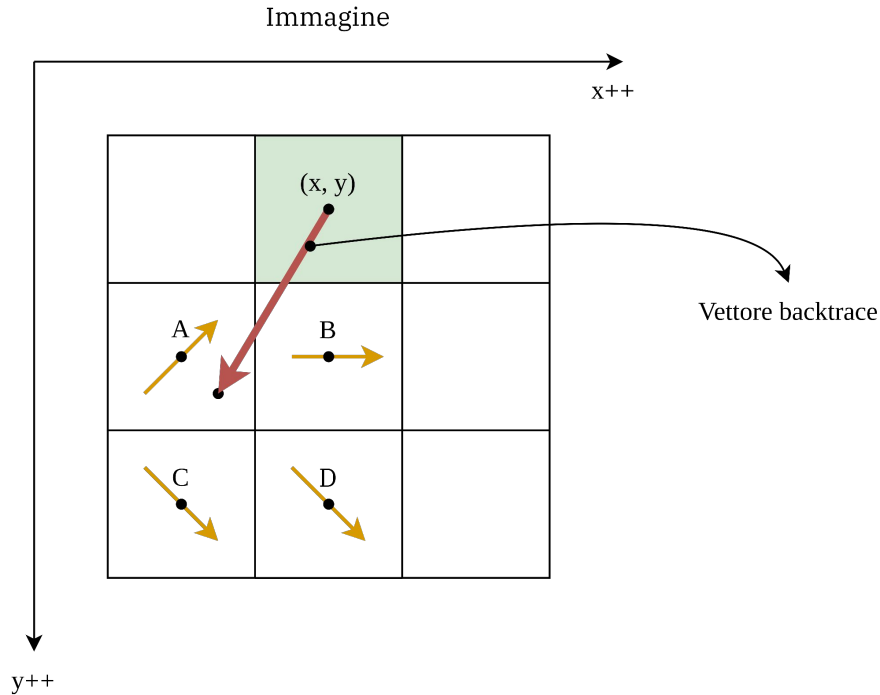
Per ogni iterazione della simulazione:

Per ogni cella di coordinate (x, y):

1. **Seguo** la velocità attuale ma a ritroso fino a trovare un certo punto nella griglia (backtrace). Questo punto è la stima della posizione di partenza delle particelle del fluido.
2. **Trovo** la velocità in quel punto. Dato che raramente il punto cade esattamente nel centro di una cella della griglia, calcolo una stima più accurata della velocità eseguendo un'interpolazione bilineare tra le velocità delle celle vicine.
3. **Trasporto** il valore ottenuto nella cella di partenza. Ovvero memorizzo il valore trovato come il nuovo valore di velocità per la cella di posizione (x,y).

$$\text{vel_next}[x, y] = \text{bilinear_interpolation}(\text{vector}(x, y) - \text{vel_current}[x, y] * \text{delta_time})$$

Calcolo avvezione della velocità - Metodo implicito

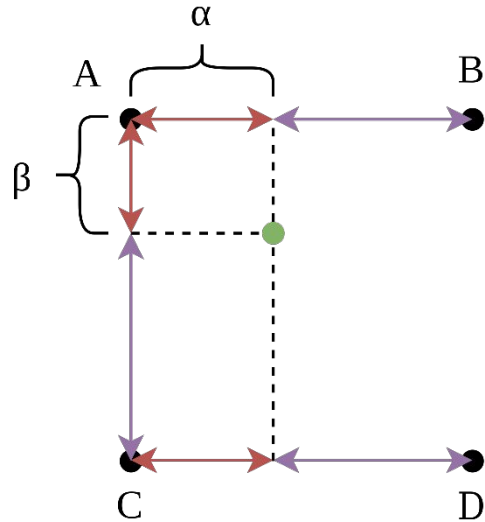


■ Vantaggi del metodo implicito:

Oltre ad essere più stabile del metodo esplicito di Eulero, questo metodo risolve anche gli altri due problemi:

Ogni thread scrive nella cella che gli è stata assegnata accedendo esclusivamente in lettura ai valori di altre celle, inoltre ogni cella viene scritta una sola volta per iterazione.

Interpolazione bilineare



$$A_B = A(1 - \alpha) + B\alpha$$

$$C_D = C(1 - \alpha) + D\alpha$$

$$A_B(1 - \beta) + C_D\beta$$

Interpolazione
orizzontale

Interpolazione
verticale

Velocità
interpolata

Interpolazione bilineare - Implementazione GLSL

```
vec4 bilinearInterpolation( vec2 pos ) {  
    // 1. Trovo le coordinate del punto A e calcolo alpha e beta  
    ivec2 pos_floor = ivec2( pos ); // floored pos  
    vec2 pos_fract = fract( pos ); // fractional part  
    // 2. Leggo i valori di velocità da A,B,C,D  
    vec4 A = imageLoad( field_current, pos_floor );  
    vec4 B = imageLoad( field_current, pos_floor + ivec2( 1, 0 ) );  
    vec4 C = imageLoad( field_current, pos_floor + ivec2( 0, 1 ) );  
    vec4 D = imageLoad( field_current, pos_floor + ivec2( 1, 1 ) );  
    // 3. Interpolo  
    return mix( mix( A, B, pos_fract.x ), mix( C, D, pos_fract.x ), pos_fract.y );  
}
```

file: /shaders/advection.comp

Calcolo avvezione della velocità - Implementazione GLSL

```
void advect( ivec2 coords, float dt ) {  
    // 1. Leggo la velocità attuale  
    vec2 actual_velocity = imageLoad( field_current, coords ).xy;  
    // 2. Seguo a ritroso la velocità  
    vec2 previous_location = coords - dt * actual_velocity;  
    // 3. Interpolo la velocità nel che raggiunto punto raggiunto  
    vec2 advected_velocity = bilinearInterpolation( previous_location ).xy;  
    // 4. Salvo il nuovo valore di velocità nella cella attuale (coords)  
    vec2 old_zw = imageLoad( field_current, coords ).zw;  
    imageStore( field_next, coords, vec4( advected_velocity.xy, old_zw ) );  
}
```

file: /shaders/advection.comp

Proiezione

Correzione del campo di velocità per garantire l'incomprimibilità:

- Dopo avvezione, diffusione e forze esterne, il campo di velocità $\mathbf{w}^3$ può presentare divergenza non nulla:

In alcune regioni del campo, il flusso di particelle in ingresso potrebbe superare quello in uscita, causando la compressione del fluido e violando così la condizione di incomprimibilità.

- Il gradiente della pressione rappresenta la componente irrotazionale del campo di velocità; è la componente responsabile della divergenza.

Soluzione: Calcolare e sottrarre il gradiente della pressione per ottenere un campo di velocità a divergenza nulla:

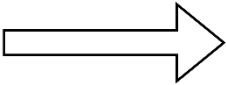
- In presenza di una differenza di pressione, un fluido tende a modificare la propria velocità per spostarsi dalla zona ad alta pressione verso quella a bassa pressione, fino ad annullare differenze di pressione. Il gradiente della pressione viene calcolato e sottratto al campo di velocità per simulare questo comportamento del fluido.

Proiezione

Per ottenere un campo con divergenza nulla, è necessario considerare la velocità indotta dalla pressione e correggere di conseguenza la velocità del fluido:

$$\underbrace{\mathbf{w}_3}_{\text{Velocità dopo avvezione, diffusione e forze esterne}} = \underbrace{\mathbf{w}_4}_{\text{Componente della velocità } \mathbf{w}_3 \text{ responsabile della divergenza}} + \underbrace{\nabla p}_{\text{Velocità indotta dalla pressione}}$$

Velocità con divergenza nulla


$$\underbrace{\mathbf{w}_4}_{\text{Velocità con divergenza nulla}} = \underbrace{\mathbf{w}_3}_{\text{Velocità che non considera il contributo della pressione}} + \underbrace{-\nabla p}_{\text{Velocità indotta dalla pressione}}$$

Semplificazione

Dal momento che lo stato $\mathbf{w}_3$ è noto per ottenere $\mathbf{w}_4$ è necessario calcolare il valore della pressione; applicando l'operatore di divergenza ad entrambi i membri dell'equazione precedente il termine $\mathbf{w}_4$ scompare, in questo modo otteniamo un'equazione per il calcolo della pressione:

$$\nabla \cdot \mathbf{w}_3 = \nabla^2 p$$



*Discretizzazione
dell'operatore di Laplace*

$$\nabla \cdot \mathbf{w}_3 = p(x+1, y) + p(x-1, y) + p(x, y+1) + p(x, y-1) - 4p(x, y)$$

Calcolo del campo della pressione

La nostra incognita è $p(x, y)$, quindi l'equazione da risolvere diventa:

$$p(x, y) = \frac{p(x+1, y) + p(x-1, y) + p(x, y+1) + p(x, y-1) - \nabla \cdot \mathbf{w}_3}{4}$$

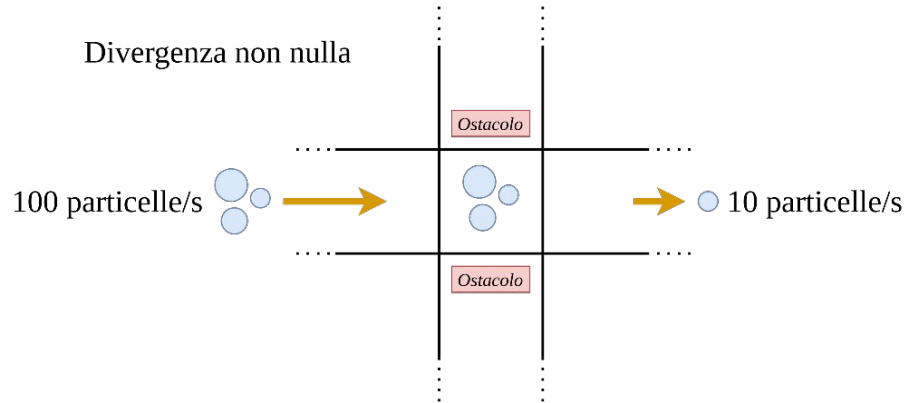
- A prima vista sembrano esserci 5 incognite e una sola equazione, in realtà abbiamo un'equazione per ogni cella del campo; quindi dato un campo di dimensioni $n \times n$ abbiamo n^2 incognite e lo stesso numero di equazioni. Questo costituisce un sistema di equazioni lineari, che verrà risolto utilizzando il **metodo di Jacobi**.

Una volta calcolata la pressione, ne calcoliamo il gradiente e lo sottraiamo a $\mathbf{w}_3$ in modo da ottenere il campo di velocità $\mathbf{w}_4$ con divergenza nulla.

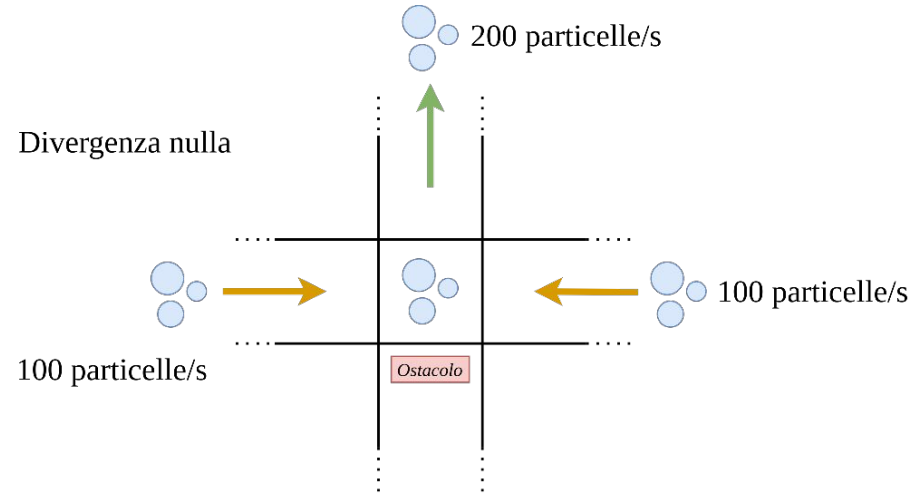
Divergenza velocità e pressione

In un fluido incompressibile con densità costante, la divergenza della velocità indica una differenza tra il flusso entrante e uscente in una cella, che si traduce in un aumento o diminuzione di pressione all'interno della cella stessa.

Divergenza negativa $\rightarrow$ aumento di pressione



$$100 \text{ particelle/s} - 10 \text{ particelle/s} = 90 \text{ particelle/s}$$



Metodo di Jacobi

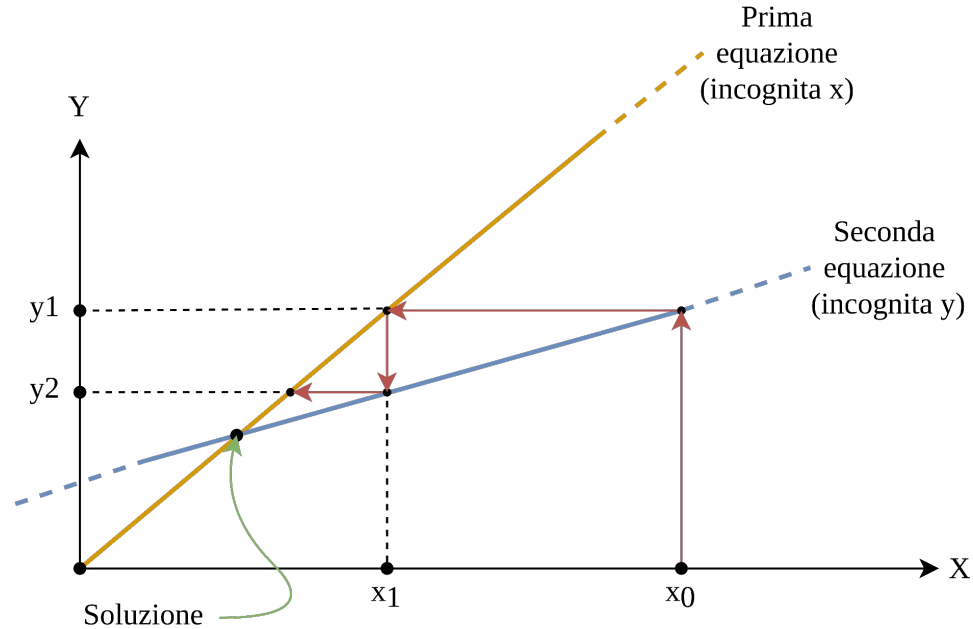
Il metodo di Jacobi è una tecnica iterativa che calcola un'approssimazione della soluzione di un sistema di equazioni lineari. Questo metodo converge verso la soluzione del sistema calcolando iterativamente i valori delle sue incognite. Questa tecnica si adatta perfettamente alle capacità di calcolo parallelo della GPU, in quanto le soluzioni approssimate ad ogni iterazione possono essere calcolate indipendentemente.

La tecnica prevede i seguenti passi:

1. Ogni incognita viene espressa in funzione delle altre.
2. Viene assegnato un valore iniziale alle incognite (di solito zero o un valore random).
3. Si risolve ciascun incognita sostituendo nella parte destra dell'equazione associata all'incognita i valori approssimati delle altre incognite, calcolati nell'iterazione precedente.
4. Si ripete il punto 3 un numero sufficiente di volte, ad ogni iterazione il sistema converge verso la soluzione esatta.

Metodo di Jacobi - Esempio

Esempio con sistema di due equazioni lineari:

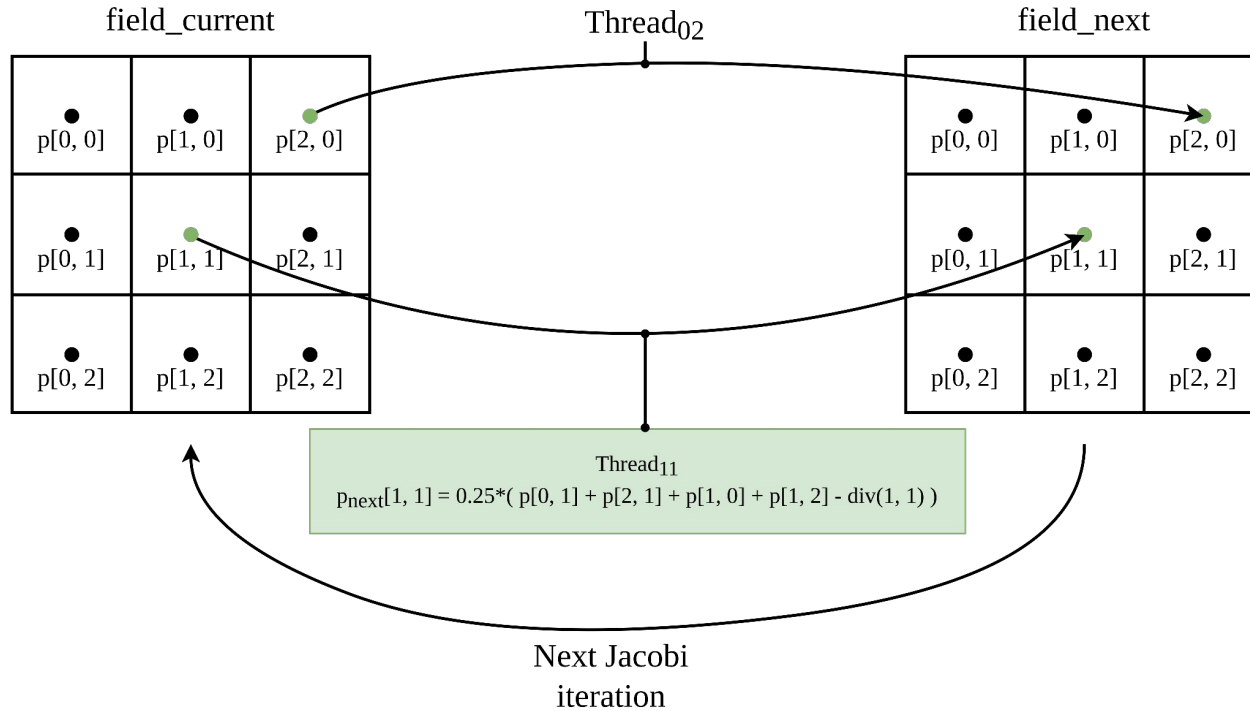


Metodo di Jacobi - Simulazione

Nel contesto della simulazione vengono impiegate **due immagini** per memorizzare i valori delle incognite; ogni **pixel** dell'immagine rappresenta il valore di un'**incognita**:

1. Ogni **thread** della GPU si occupa di risolvere una **singola incognita**; il thread legge dall'immagine i valori approssimati delle altre incognite nei pixel vicini e li usa per risolvere l'equazione.
2. Il nuovo valore dell'incognita viene memorizzato nel pixel corrispondente della **seconda** immagine.
3. Al termine di ogni iterazione, le due immagini vengono **scambiate** in modo che i valori aggiornati delle incognite siano utilizzati come input dell'iterazione successiva.

Metodo di Jacobi - Simulazione e parallelismo



Calcolo della divergenza della velocità - Implementazione GLSL

```
float velocityDivergency( ivec2 coords ) {  
    // 1. Leggo i valori di velocità dei vicini (L, R, T, B)  
  
    vec2 L = imageLoad( field_current, coords - ivec2(1, 0) ).xy;  
    vec2 R = imageLoad( field_current, coords + ivec2(1, 0) ).xy;  
    vec2 T = imageLoad( field_current, coords - ivec2(0, 1) ).xy;  
    vec2 B = imageLoad( field_current, coords + ivec2(0, 1) ).xy;  
  
    // 2. Calcolo la divergenza della velocità  
  
    return ( R.x - L.x ) / 2 + ( B.y - T.y ) / 2;  
}
```

file: /shaders/jacobi_pressure.comp

Calcolo del campo della pressione - Implementazione GLSL

```
void jacobiSolverPressure ivec2 coords ) {  
    // 1. Leggo i valori di pressione dei vicini (L, R, T, B)  
    float L = imageLoad( field_current, coords - ivec2(1, 0) ).z;  
    float R = imageLoad( field_current, coords + ivec2(1, 0) ).z;  
    float T = imageLoad( field_current, coords - ivec2(0, 1) ).z;  
    float B = imageLoad( field_current, coords + ivec2(0, 1) ).z;  
  
    // 2. Calcolo la divergenza della velocità w3  
    float vel_div = velocityDivergency( coords, 1.0f, 1.0f );  
  
    // 3. Calcolo del nuovo valore di pressione  
    float p_new = ( ( R + L + B + T ) - vel_div ) / 4.0f;  
  
    // 4. Salvo il nuovo valore nel campo next  
    vec4 tmp = imageLoad( field_next, coords );  
    tmp.z = p_new;  
    imageStore( field_next, coords, tmp );  
}  
file: /shaders/jacobi_pressure.comp
```

Calcolo velocità finale - Implementazione GLSL

```
vec2 pressureGradient( ivec2 coords ) {  
  
    // 1. Leggo i valori di pressione dei vicini (L, R, T, B)  
  
    float L = imageLoad( field_current, coords - ivec2(1, 0) ).z;  
    float R = imageLoad( field_current, coords + ivec2(1, 0) ).z;  
    float T = imageLoad( field_current, coords - ivec2(0, 1) ).z;  
    float B = imageLoad( field_current, coords + ivec2(0, 1) ).z;  
  
    // 2. Calcolo il gradiente della pressione  
  
    return vec2( ( R - L ) / 2, ( B - T ) / 2 );  
  
}  
  
file: /shaders/remove_divergency.comp
```

```
void main() {  
  
    // 1. Posizione attuale  
  
    ivec2 coords = ivec2(gl_GlobalInvocationID.xy);  
  
    // 2. Sottraggo il gradiente della pressione a w3  
  
    vec4 w3 = imageLoad( field_current, coords );  
    vec2 w4 = w3.xy - pressureGradient( coords );  
  
    // 3. Memorizzo il risultato  
  
    imageStore(field_current, coords, vec4( w4.xy, w3.zw ));  
  
}  
  
file: /shaders/remove_divergency.comp
```

Calcolo della diffusione - Metodo esplicito

Per capire intuitivamente come varia la velocità nel tempo a causa della diffusione possiamo basarci sulla soluzione ottenuta attraverso il **metodo esplicito** di Eulero:

$$\frac{\mathbf{w}_{next}(x, y) - \mathbf{w}_{current}(x, y)}{\Delta t} = k \nabla^2 \mathbf{w}_{current}(x, y)$$

(Considero solo l'effetto della diffusione)



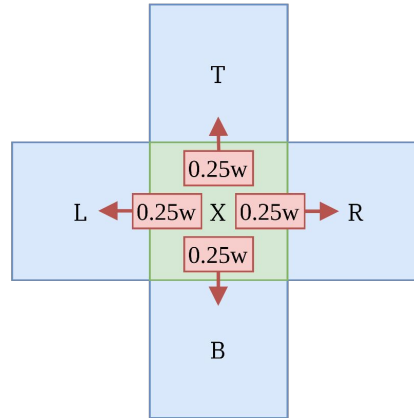
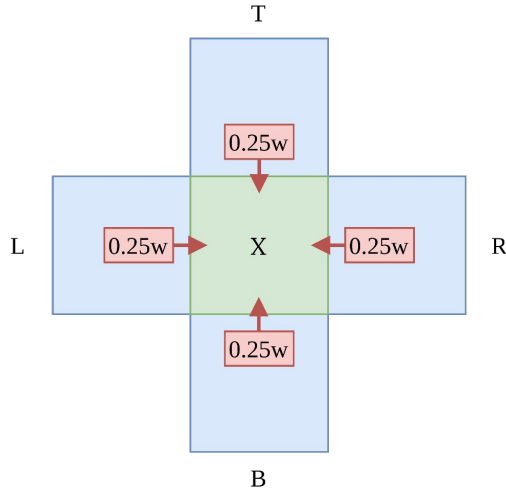
$$\mathbf{w}_{next} = \mathbf{w}_{current} + \Delta t \left(k \nabla^2 \mathbf{w}_{current} \right)$$



$$\mathbf{w}_n(x, y) = \mathbf{w}_c(x, y) + k\Delta t \left(\mathbf{w}_c(x+1, y) + \mathbf{w}_c(x-1, y) + \mathbf{w}_c(x, y+1) + \mathbf{w}_c(x, y-1) - 4\mathbf{w}_c(x, y) \right)$$

Calcolo della diffusione - Metodo esplicito

Per semplicità consideriamo $\Delta t = 1$ e $k = 1/4$.



- Ogni cella **cede** una parte di velocità a ciascun vicino e **riceve** una parte da ognuno di essi. Nel tempo, grazie a questo processo di scambio, la **velocità tende a uniformarsi** lungo il campo.
- **Problema:** Esattamente come nel caso del calcolo dell'avvezione questo metodo diventa **instabile** quando il passo temporale Δt è troppo grande.

Calcolo della diffusione - Metodo implicito

La soluzione implicita proposta da Stam è più stabile ma il termine Laplaciano è calcolato al tempo futuro:

$$\mathbf{w}_3(x, y) = \mathbf{w}_2(x, y) + \Delta t \nu \nabla^2 \mathbf{w}_3(x, y)$$



Discretizzando l'operatore di Laplace e risolvendo rispetto $\mathbf{w}_3$

$$\mathbf{w}_3(x, y) = \frac{\mathbf{w}_2(x, y) + \frac{\nu}{4} \cdot \Delta t \cdot (\mathbf{w}_3(x+1, y) + \mathbf{w}_3(x-1, y) + \mathbf{w}_3(x, y+1) + \mathbf{w}_3(x, y-1))}{1 + \nu \cdot \Delta t}$$

Come nel caso della proiezione, questo rappresenta un sistema di equazioni lineari; per ogni cella del campo è necessario calcolare l'incognita $\mathbf{w}_3(x, y)$ che dipende dal valore precedente del campo $\mathbf{w}_2(x, y)$ e dal valore futuro delle 4 celle adiacenti, anch'esse incognite. Si ottiene, quindi, un sistema con n^2 incognite e lo stesso numero di equazioni che, anche in questo caso, verrà risolto con il **metodo di Jacobi**.

Calcolo diffusione della velocità - Implementazione GLSL

```
void jacobiSolverDiffusion( ivec2 coords, float diffusion_rate, float dt ) {  
    // 1. Leggo i valori di velocità dei vicini (L, R, T, B)  
  
    vec2 L = imageLoad( field_current, coords - ivec2(1, 0) ).xy;  
    vec2 R = imageLoad( field_current, coords + ivec2(1, 0) ).xy;  
    vec2 T = imageLoad( field_current, coords - ivec2(0, 1) ).xy;  
    vec2 B = imageLoad( field_current, coords + ivec2(0, 1) ).xy;  
    vec2 X = imageLoad( field_current, coords ).xy;  
  
    float D = diffusion_rate * dt; // diffusion_rate = viscosity factor  
  
    // 2. Calcolo del nuovo valore di velocità  
  
    vec2 new_vel = ( X + D * 0.25f * ( R + L + B + T ) ) / ( 1.0f + D );  
  
    // 3. Salvataggio del nuovo valore nel campo next  
  
    vec2 old_zw = imageLoad( field_next, coords ).zw;  
    imageStore( field_next, coords, vec4(new_vel, old_zw) );  
}  
file: /shaders/jacobi_diffusion.comp
```

Implementazione con Vulkan

- Motore di simulazione scritto in C++.
- **API Vulkan**: un'API grafica e di calcolo moderna e di basso livello, progettata per essere utilizzata in ambienti ad alte prestazioni; supporta i *compute-shader*, ovvero programmi che possono essere eseguiti in parallelo dalla GPU per il calcolo general-purpose.
- I compute-shader per la simulazione sono scritti in linguaggio *OpenGL Shading Language* (GLSL); il motore compila gli shaders tramite il compilatore GLSL di Khronos: “*Glslang*”.
- La simulazione utilizza tre immagini allocate nella memoria della GPU (VRAM):
 - Due immagini vengono usate per il calcolo del campo di velocità e di pressione del fluido, la terza viene utilizzata per memorizzare il colore del risultato che dovrà essere visualizzato a schermo. Il motore copia la terza immagine in una delle immagini della *swapchain* ogni volta che deve essere visualizzato un frame.
 - Le immagini sono a quattro componenti float di 32 bit.

Swapchain

Una **swapchain** è un meccanismo che gestisce una serie di immagini; viene usata dalla GPU e dalle API grafiche per sincronizzare i frame renderizzati con il refresh-rate dello schermo.

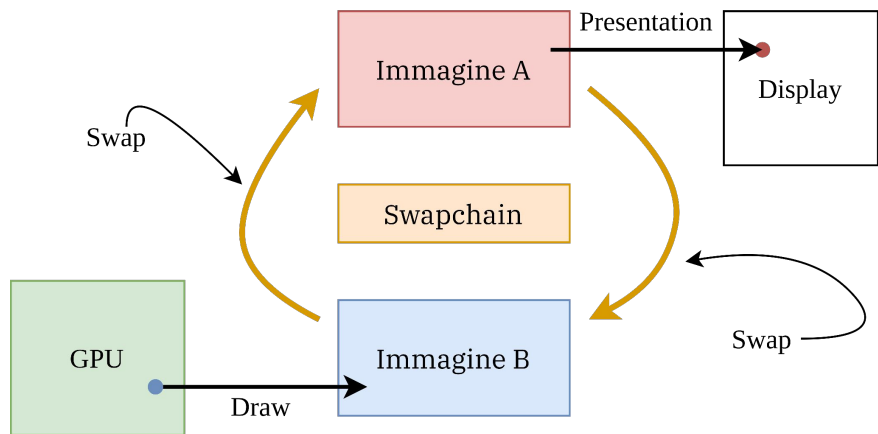
Una swapchain contiene almeno due immagini (double-buffer):

- La **prima immagine** è usata per **presentare** un frame generato dalla GPU sullo schermo.
- La **seconda immagine** è utilizzata dalla GPU per **renderizzare** il frame successivo.
- Ogni volta che un frame è visualizzato a schermo le due immagini vengono **scambiate** (swapping) seguendo una regola di sincronizzazione (presentation mode).

Se ci fosse una sola immagine nella swapchain, la GPU non potrebbe iniziare a renderizzare un nuovo frame finché l'immagine corrente non fosse completamente visualizzata sullo schermo, causando così un rallentamento del rendering.

Swapchain

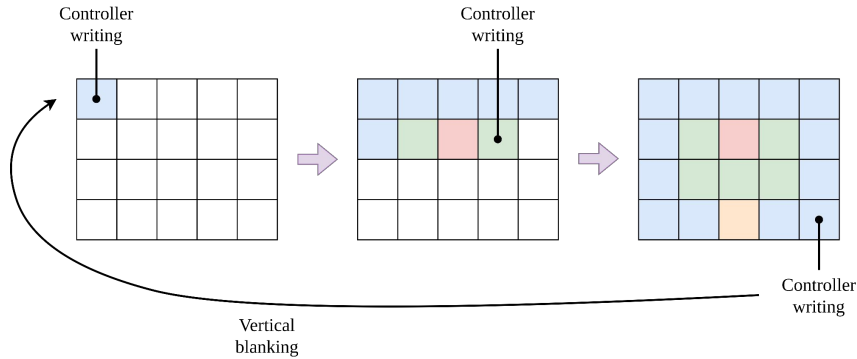
Esistono diverse modalità di presentazione delle immagini in base alle necessità dell'applicazione.



- Il motore di simulazione sfrutta una **coda FIFO** per gestire le richieste di presentazione in sospeso. La swapchain attende la GPU nel caso in cui la coda si svuoti. Questa modalità di presentazione attende il successivo **intervallo di blanking verticale** per aggiornare l'immagine corrente:

1. Evita artefatti di sincronizzazione come lo **screen tearing** (disallineamento visivo tra un frame e il successivo).
2. Lo svantaggio è che la velocità della simulazione è limitata superiormente dal frame rate dello schermo.

Intervallo di blanking verticale e screen tearing



- **Intervallo di blanking verticale:** In uno schermo l'immagine è disegnata riga per riga, dall'alto verso il basso, una volta che l'immagine è completata il controller del display deve "tornare" alla prima riga di pixel dello schermo per iniziare il disegno del frame successivo. Questo intervallo di tempo è chiamato, appunto, intervallo di blanking verticale.
- **Screen tearing:** un nuovo frame viene scritto nel framebuffer mentre lo schermo sta ancora leggendo il frame precedente.

Command Buffer

Un **command buffer** è un oggetto impiegato per registrare i comandi che verranno successivamente inviati nella coda di esecuzione di una GPU.

Questi comandi includono per esempio:

- Comandi per gestire pipeline e set di descrittori
- Comandi per il calcolo (come il dispatch di compute-shader)
- Comandi per sincronizzare risorse
- Comandi per copiare buffer e immagini

Un aspetto fondamentale dei command buffer è l'**esecuzione asincrona**: una volta inviati a una coda di esecuzione della GPU la loro esecuzione avviene in parallelo con le operazioni della CPU; è quindi necessaria una sincronizzazione esplicita tra CPU e GPU. In ultima analisi, un command buffer è il mezzo attraverso cui la CPU comunica alla GPU cosa fare.

Stati del Command Buffer

In generale un command buffer può trovarsi in uno dei seguenti stati:

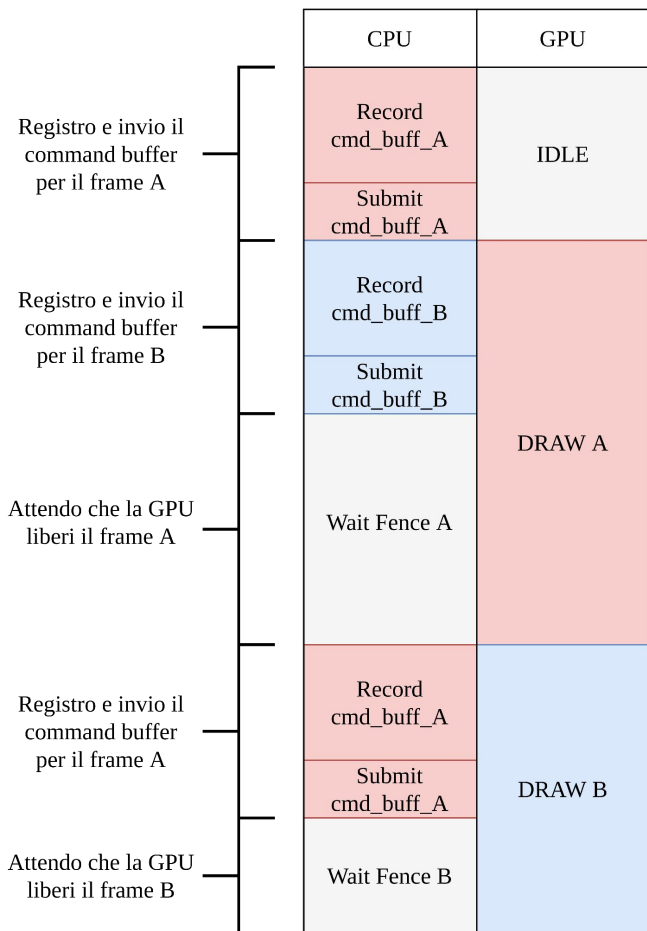
1. **Stato iniziale:** il command buffer è appena stato allocato tramite un **VkCommandPool**, che gestisce la memoria per questi oggetti.
2. **Stato di registrazione:** l'istruzione **vkBeginCommandBuffer** cambia lo stato del command buffer dallo stato iniziale allo stato di registrazione; ora è possibile registrare sul command buffer dei comandi per la GPU.
3. **Stato di esecuzione:** l'istruzione **vkEndCommandBuffer** termina lo stato di registrazione e sposta il command buffer nello stato di esecuzione; ora il command buffer è pronto per essere inviato a una coda per l'esecuzione sulla GPU (**vkQueueSubmit**).

Sincronizzazione CPU - GPU

VkFence: è una primitiva di sincronizzazione utilizzata per segnalare alla CPU quando la GPU ha completato un insieme di operazioni. Alcune operazioni Vulkan accettano un oggetto `VkFence` come parametro opzionale. Un esempio è il comando `vkQueueSubmit` che invia alla GPU le operazioni registrate in un command buffer associandole a una fence:

- Quando le operazioni inviate alla coda di esecuzione della GPU sono state completate la fence viene portata nello stato “**segnalata**”.
- Attraverso il comando `vkWaitForFences` la CPU attende che la fence si trovi nello stato “segnalata” prima di procedere con l’esecuzione delle istruzioni successive.

Questo meccanismo serve a sincronizzare il loop principale gestito dalla CPU con le operazioni eseguite dalla GPU (come la copia di immagini o l’esecuzione dei compute-shader), garantendo che la CPU non proceda a modificare o utilizzare risorse che la GPU sta ancora elaborando.



PSEUDOCODICE SINCRONIZZAZIONE CPU ⇔ GPU (double-buffer)

```

drawLoop()
{
    while(true)
    {
        // 1. Ottieni la fence associata al frame corrente
        VkFence fence = currentFrame.fence;

        // 2. Attendi che la GPU completi le operazioni associate al frame
        waitForFence(fence);

        // 3. Imposta la fence come "non segnalata" in modo da poterla riutilizzare
        resetFence(fence);

        // 4. Registra le nuove operazioni che la GPU dovrà eseguire
        recordOperations(commandBuffer, someOperations);

        // 5. Invia nuove operazioni associandole alla fence
        submitToGPUQueue(commandBuffer, fence);

        // 6. Passa al frame successivo (double-buffering)
        swapCurrentFrame();
    }
}

```

=> A e B immagini della swapchain

Nota: Quando si itera per la prima volta su un frame, la fence associata a quel frame deve già trovarsi nello stato "segnalata", altrimenti il thread della CPU si blocca, in attesa della fence.

Sincronizzazione GPU - GPU

In aggiunta alle fence (sincronizzazione CPU-GPU), è necessario un meccanismo di sincronizzazione GPU-GPU.

VkSemaphore: Primitiva di sincronizzazione utilizzata per definire l'ordine di esecuzione delle operazioni svolte dalla GPU.

Nel motore vengono utilizzati due semafori per sincronizzare il processo di rendering con la swapchain:

1. **Semaforo per l'acquisizione di un'immagine dalla swapchain:**

I comandi di rendering attendono la disponibilità di un'immagine della swapchain.

2. **Semaforo per il completamento del rendering:**

Un'immagine viene inviata alla coda di presentazione della swapchain solo dopo che il rendering su quell'immagine è stato completato.

```

drawLoop()
{
    while(true)
    {
        // 1. Ottieni le risorse associate al frame corrente:

        VkFence          fence          = currentFrame.fence;
        VkCommandBuffer  commandBuffer  = currentFrame.commandBuffer;
        VkSemaphore      swapchainSemaphore = currentFrame.swapchainSemaphore;
        VkSemaphore      renderSemaphore  = currentFrame.renderSemaphore;

        // 2. Attendi che la GPU completi eventuali operazioni associate al frame corrente:
        waitForFence(fence);

        // 3. Reimposta la fence (imposta su "non segnalata"):
        resetFence(fence);

        // 4. Determina quale immagine della swapchain sarà utilizzata per il rendering del frame corrente e salva l'indice.
        // Utilizza il comando acquireNextImage per acquisire tale immagine dalla swapchain quando sarà disponibile.
        // Durante questa operazione, il semaforo swapchainSemaphore viene associato all'acquisizione dell'immagine;
        // il semaforo verrà automaticamente segnalato quando l'immagine diventerà disponibile.
        // Questo garantisce che il rendering del frame inizi solo dopo che l'immagine della swapchain
        // è stata effettivamente acquisita:

        int swapchainImageIndex = acquireNextImage(swapchainSemaphore);

        // 5. Inizia la registrazione dei comandi sul command buffer:
        beginCommandBuffer(commandBuffer);

        // 6. Registra sul command buffer i comandi per la simulazione:
        computeSimulationStep(commandBuffer);
    }
}

```





```
// 7. Registra i comandi per copiare il risultato della simulazione sull'immagine della swapchain:
copyImageToImage(commandBuffer, computedImage, swapchainImageIndex);

// 8. Termina la registrazione dei comandi:
endCommandBuffer(commandBuffer);

// 9. Imposta il semaforo in modo che venga segnalato solo dopo che tutte le operazioni
// grafiche sul frame corrente sono state completate:

renderSemaphore.stage = VK_PIPELINE_STAGE_ALL_GRAPHICS_BIT;

// 10. Assicura che la GPU non inizi a scrivere sull'immagine della swapchain finché questa non è disponibile:
swapchainSemaphore.stage = VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT_KHR;

// 11. Invia le operazioni da eseguire alla coda di esecuzione della GPU, associandole alla fence
// e ai due semafori del frame:

submitToGPUQueue(commandBuffer, fence, swapchainSemaphore, renderSemaphore);

// 12. Presenta l'immagine della swapchain quando il rendering è completato,
// ovvero quando renderSemaphore viene "segnalato".

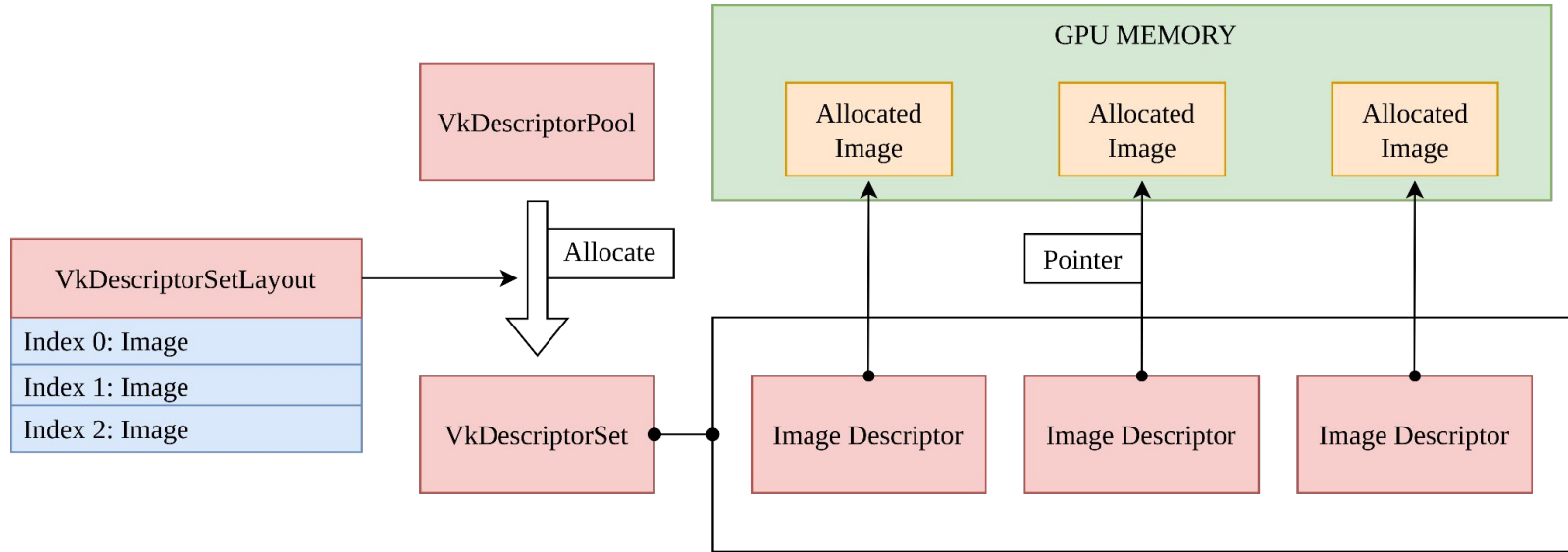
presentQueue(swapchainImageIndex, renderSemaphore);

// 13. Passa al frame successivo (double-buffering):
swapCurrentFrame();
}
}
```

Descriptor e DescriptorSet

- Un singolo *descriptor* può essere immaginato come il riferimento o come il puntatore a una risorsa di uno shader, come per esempio un'immagine, un buffer o una struttura per l'accelerazione dei calcoli (esempio: BVH ray tracing)
- I descriptor non contengono direttamente i dati della risorsa, ma piuttosto le informazioni necessarie per accedere a questi dati nella memoria della GPU.
- I descriptor sono organizzati in *descriptor-set* che vengono collegati allo stage (pipeline) di uno shader durante la fase di registrazione dei comandi sul command buffer.
- Ogni descriptor-set contiene quindi uno o più descriptor ciascuno dei quali è il riferimento a una risorsa diversa allocata nella memoria della GPU, un descriptor-set viene allocato da un **VkDescriptorPool**, che gestisce la memoria per questo tipo di oggetti.

Descriptor e DescriptorSet



Nota: Vulkan non consente di associare singole risorse agli shader; le risorse devono essere raggruppate in set.

DescriptorSet e Shader

Le risorse referenziate da un descriptor-set sono accessibili nello shader indicando l'indice che identifica il descriptor-set e l'indice di binding che identifica uno specifico descriptor all'interno di quel set:

```
layout(rgba32f, set = 0, binding = 2) uniform image2D image;
```

Indica che la risorsa image è un'immagine 2D con formato rgba32f, accessibile tramite il **descriptor di indice 2** del **descriptor-set di indice 0**.

DescriptorSet - Simulazione

Nel motore di simulazione tutti i descriptor-set usano lo stesso layout:

Ogni descriptor set contiene tre descriptor, ogni descriptor punta a un'immagine diversa (allocata nella memoria della GPU); tutti i descriptor set puntano allo stesso insieme di immagini.

Queste tre immagine vengono impiegate come segue:

1. La prima immagine memorizza lo **stato attuale** del campo di velocità e di pressione del fluido.
2. La seconda immagine memorizza il **prossimo stato** del campo di velocità e di pressione del fluido.
3. La terza immagine memorizza il **colore** del risultato della simulazione; quest'immagine verrà poi copiata nella swapchain per essere presentata a schermo.

DescriptorSet - Simulazione

Il motore di simulazione utilizza i descriptor-set in **due modi**:

1. Per gli shaders che calcolano la soluzione di un sistema di equazioni lineari tramite il **metodo di Jacobi**, il motore **alterna due descriptor-set**. Questi descriptor-set puntano allo stesso insieme di immagini, ma gli indici delle prime due immagini sono **invertiti**. Ad ogni iterazione del metodo di Jacobi, il motore scambia il descriptor-set collegato allo shader, in questo modo l'output dello step n diventa l'input dello step n+1.
2. Per gli altri shaders il motore di simulazione utilizza sempre lo stesso descriptor set.

JacobiPipeline: rappresenta una pipeline di calcolo associata al metodo di Jacobi. Una pipeline di calcolo contiene:

1. Il codice del compute-shader.
2. Il layout della pipeline, che definisce i **descriptor-set**, le **push constant**, e altre risorse che possono essere legate allo shader.

Metodo di Jacobi - Pseudocodice Vulkan

```
runJacobiSolver() {  
    bool toggle = false;  
  
    for (int i = 0; i < iterations; ++i)  
    {  
        // 1. Seleziona il descriptorSet da usare in questa iterazione  
        if (toggle)  
            bindDescriptorSets(commandBuffer, jacobiPipeline, descriptorSet0);  
        else  
            bindDescriptorSets(commandBuffer, jacobiPipeline, descriptorSet1);  
  
        // 2. Avvia la computazione del compute shader  
        dispatch(commandBuffer);  
  
        // 3. Sincronizza le operazioni di lettura e scrittura tra le iterazioni  
        memoryBarrier(command_buffer);  
  
        // Scambia i descriptor sets nella prossima iterazione  
        toggle = !toggle;  
    }  
}
```

Nota: L'istruzione `memoryBarrier` garantisce che l'esecuzione del compute shader associato all'iterazione `n` inizi solo dopo che tutte le operazioni di scrittura effettuate nell'iterazione `n-1` siano state completate.

PushConstants

Le **push constants** permettono l'invio di una piccola quantità di dati di sola lettura a uno shader in modo semplice e performante. Si tratta di un meccanismo di comunicazione unidirezionale che consente l'invio di dati dalla CPU alla GPU.

Sono particolarmente utili per i dati che cambiano **frequentemente**, come lo stato dell'input o il tempo.

```
struct ComputePushConstants
{
    uint32_t    mouse_down    {}; // un intero senza segno che indica se il mouse è stato premuto.
    uint32_t    time_elapsed  {}; // tempo trascorso tra due iterazioni della simulazione (numero ms).
    glm::ivec2  mouse_pos     {}; // vettore utilizzato per inviare la posizione del mouse.
};
```

I dati contenuti nella struttura vengono successivamente incorporati nel command buffer e resi accessibili allo shader. In particolare il motore passa a tutti i compute-shader lo stesso insieme di dati per ogni step di simulazione.

Conclusioni

Questo progetto mi ha permesso di raggiungere alcuni obiettivi significativi:

1. Acquisito competenze nell'utilizzo delle API Vulkan.
2. Imparato ad utilizzare GLSL per scrivere programmi GPU-based.
3. Approfondito il funzionamento di un motore di rendering.
4. Appreso diverse tecniche per approssimare e calcolare funzioni complesse.
5. Approcciato le equazioni di Navier-Stokes e il loro significato.
6. Sviluppato una comprensione più approfondita di come un fenomeno complesso, come il moto di un fluido, possa essere simulato in tempo reale.

Riferimenti

Joe Stam. 1999. "*Stable Fluids*." In Proceedings of SIGGRAPH 1999:

https://www.researchgate.net/publication/2486965_Stable_Fluids

Mark J. Harris. 2004. "*Fast Fluid Dynamics Simulation on the GPU*". In GPU gems 3, chapter 38:

<https://developer.nvidia.com/gpugems/gpugems/part-vi-beyond-triangles/chapter-38-fast-fluid-dynamics-simulation-gpu>

Shahriar Shahrabi. 2021 "*Gentle Introduction to Realtime Fluid Simulation for Programmers and Technical Artists*":

<https://shahriarshahrabi.medium.com/gentle-introduction-to-fluid-simulation-for-programmers-and-technical-artists-7c0045c40bac>

Victor Blanco. 2020. "*Vulkan Guide*":

<https://vkguide.dev/>